

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)

QUESTIONS: 10

Total

Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I **T.CHANDRALEKHA(192472290)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

```
class Book {  
    private int bookId;  
    private String title;  
    private boolean issued;  
    Book(int bookId, String title) {  
        this.bookId = bookId;  
        this.title = title;  
        this.issued = false;  
    }  
}
```

```
void issueBook() {  
    issued = true;  
    System.out.println("Book Issued");  
}  
  
void returnBook() {  
    issued = false;  
    System.out.println("Book Returned");  
}  
  
void display() {  
    System.out.println("Book ID: " + bookId);  
    System.out.println("Title: " + title);  
    System.out.println("Status: " + (issued ? "Issued" : "Available"));  
}  
  
public static void main(String[] args) {  
    Book b = new Book(101, "Java Programming");  
    b.display();  
    b.issueBook();  
    b.display();  
    b.returnBook();  
    b.display();  
}  
}
```

Book.java	Run	Output
<pre> 28 public static void main(String[] args) { 29 Book b = new Book(101, "Java Programming" 30); 31 b.display(); 32 b.issueBook(); 33 b.display(); 34 b.returnBook(); 35 b.display(); 36 } </pre>	Run	<pre> Book ID: 101 Title: Java Programming Status: Available Book Issued Book ID: 101 Title: Java Programming Status: Issued Book Returned Book ID: 101 Title: Java Programming Status: Available </pre>

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply) class Employee {

String name;

int id;

Employee(String name, int id) {

 this.name = name;

 this.id = id;

}

double calculateSalary() {

 return 0;

}

void display() {

 System.out.println("ID: " + id);

 System.out.println("Name: " + name);

 System.out.println("Salary: " + calculateSalary());

```
        System.out.println();
    }

    public static void main(String[] args) {
        Employee e1 = new PermanentEmployee("Rahul", 101, 50000);
        Employee e2 = new ContractEmployee("Priya", 102, 100, 500);

        e1.display();
        e2.display();
    }
}
```

```
class PermanentEmployee extends Employee {
    double basicSalary;

    PermanentEmployee(String name, int id, double basicSalary) {
        super(name, id);
        this.basicSalary = basicSalary;
    }
}
```

```
@Override
double calculateSalary() {
    return basicSalary + basicSalary * 0.2;
}
}
```

```
class ContractEmployee extends Employee {
    int hours;
    double rate;
```

```

ContractEmployee(String name, int id, int hours, double rate) {
    super(name, id);
    this.hours = hours;
    this.rate = rate;
}

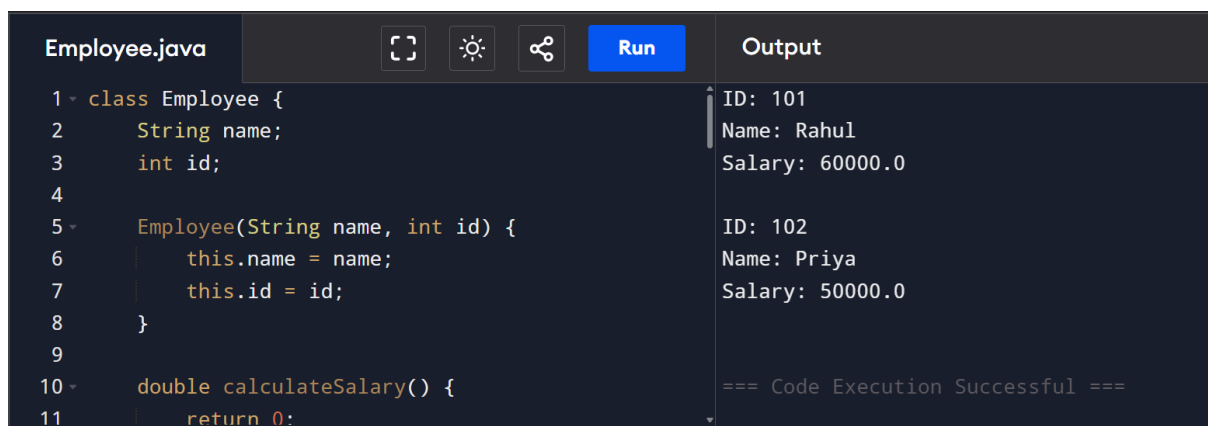
```

@Override

```

double calculateSalary() {
    return hours * rate;
}
}

```



The screenshot shows an IDE window titled 'Employee.java'. The code defines a class 'Employee' with attributes 'name' and 'id', a constructor, and a 'calculateSalary()' method. The 'Run' button is highlighted. The 'Output' pane on the right shows the execution results for two instances: ID: 101, Name: Rahul, Salary: 60000.0 and ID: 102, Name: Priya, Salary: 50000.0. A message at the bottom states '=== Code Execution Successful ==='.

```

Employee.java
class Employee {
    String name;
    int id;
    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }
    double calculateSalary() {
        return 0;
    }
}

```

Output

```

ID: 101
Name: Rahul
Salary: 60000.0
ID: 102
Name: Priya
Salary: 50000.0
=== Code Execution Successful ===

```

3. Vehicle Rental System using Runtime Polymorphism Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

```

class Vehicle {
    String vehicleName;

    Vehicle(String vehicleName) {
        this.vehicleName = vehicleName;
    }
}

```

```
}
```

```
double calculateRent(int days) {  
    return 0;  
}
```

```
void displayBill(int days) {  
    System.out.println("Vehicle : " + vehicleName);  
    System.out.println("Days    : " + days);  
    System.out.println("Rent    : " + calculateRent(days));  
    System.out.println();  
}
```

```
public static void main(String[] args) {
```

```
    Vehicle[] vehicles = new Vehicle[3];
```

```
    vehicles[0] = new Car("Car");
```

```
    vehicles[1] = new Bike("Bike");
```

```
    vehicles[2] = new Bus("Bus");
```

```
    int days = 5;
```

```
    for (Vehicle v : vehicles) {
```

```
        v.displayBill(days);
```

```
    }
```

```
}
```

```
}
```

```
class Car extends Vehicle {
```

```
    Car(String name) {
```

```
        super(name);
```

```
    }
```

```
    @Override
```

```
    double calculateRent(int days) {
```

```
        return days * 1000;
```

```
    }
```

```
}
```

```
class Bike extends Vehicle {
```

```
    Bike(String name) {
```

```
        super(name);
```

```
    }
```

```
    @Override
```

```
    double calculateRent(int days) {
```

```
        return days * 300;
```

```
    }
```

```
}
```

```
class Bus extends Vehicle {
```

```
    Bus(String name) {
```

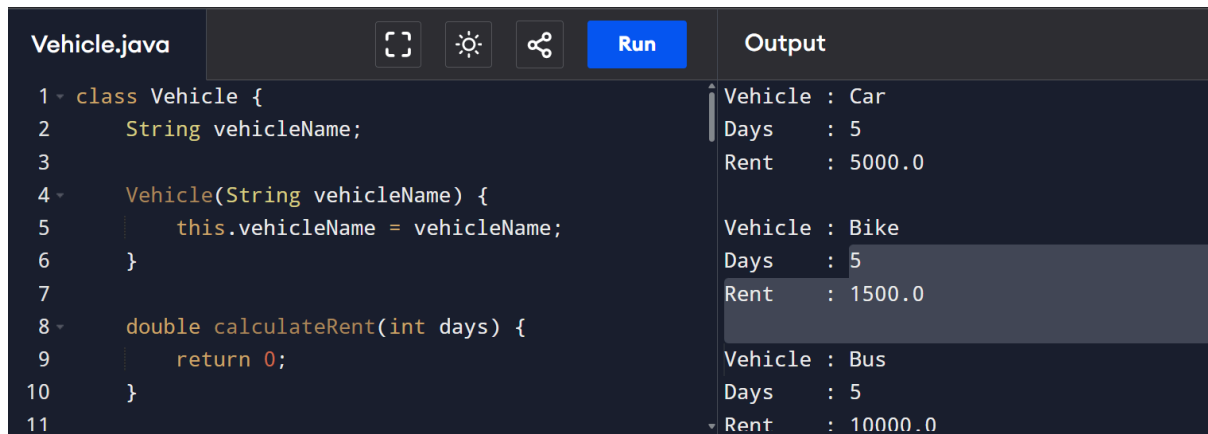
```
        super(name);
```

```
    }
```

```

@Override
double calculateRent(int days) {
    return days * 2000;
}
}

```



File	Code	Output
Vehicle.java	<pre> 1 class Vehicle { 2 String vehicleName; 3 4 Vehicle(String vehicleName) { 5 this.vehicleName = vehicleName; 6 } 7 8 double calculateRent(int days) { 9 return 0; 10 } 11 </pre>	<pre> Vehicle : Car Days : 5 Rent : 5000.0 Vehicle : Bike Days : 5 Rent : 1500.0 Vehicle : Bus Days : 5 Rent : 10000.0 </pre>

4. Bank Account System using Data Hiding Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

```

class BankAccount {
    // Private data members

    private int accountNumber;

    private String holderName;

    private double balance;

    // Constructor

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;
    }
}

```



```
}
```

```
// Deposit method
```

```
void deposit(double amount) {  
    if (amount > 0) {  
        balance += amount;  
        System.out.println("Deposited: " + amount);  
    } else {  
        System.out.println("Invalid Deposit Amount");  
    }  
}
```

```
// Withdrawal method
```

```
void withdraw(double amount) {  
    if (amount > 0 && amount <= balance) {  
        balance -= amount;  
        System.out.println("Withdrawn: " + amount);  
    } else {  
        System.out.println("Invalid Withdrawal");  
    }  
}
```

```
// Balance inquiry
```

```
void showBalance() {  
    System.out.println("Account Number: " + accountNumber);  
    System.out.println("Holder Name: " + holderName);  
    System.out.println("Balance: " + balance);  
    System.out.println();  
}
```

```

// Main method

public static void main(String[] args) {

    BankAccount account = new BankAccount(101, "Rahul", 5000);

    account.showBalance();

    account.deposit(2000);
    account.showBalance();

    account.withdraw(3000);
    account.showBalance();

    account.withdraw(6000); // Invalid
    account.deposit(-500); // Invalid
}
}

```

Program	Run	Output
BankAccount.java <pre> 1 class BankAccount { 2 // Private data members 3 private int accountNumber; 4 private String holderName; 5 private double balance; 6 7 // Constructor 8 BankAccount(int accountNumber, String holderName, double balance) { 9 this.accountNumber = accountNumber; 10 this.holderName = holderName; </pre>	Run	<pre> Account Number: 101 Holder Name: Rahul Balance: 5000.0 Deposited: 2000.0 Account Number: 101 Holder Name: Rahul Balance: 7000.0 Withdrawn: 3000.0 Account Number: 101 </pre>

5. University Admission System using Method Overloading Create an Admission class that overloads the method calculateFee() for: • Undergraduate students, • Postgraduate students,

- Scholarship students. Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

```
class Admission {

    // Undergraduate fee
    void calculateFee() {
        System.out.println("Undergraduate Fee: ₹50000");
    }

    // Postgraduate fee
    void calculateFee(String course) {
        System.out.println("Postgraduate Fee: ₹70000");
    }

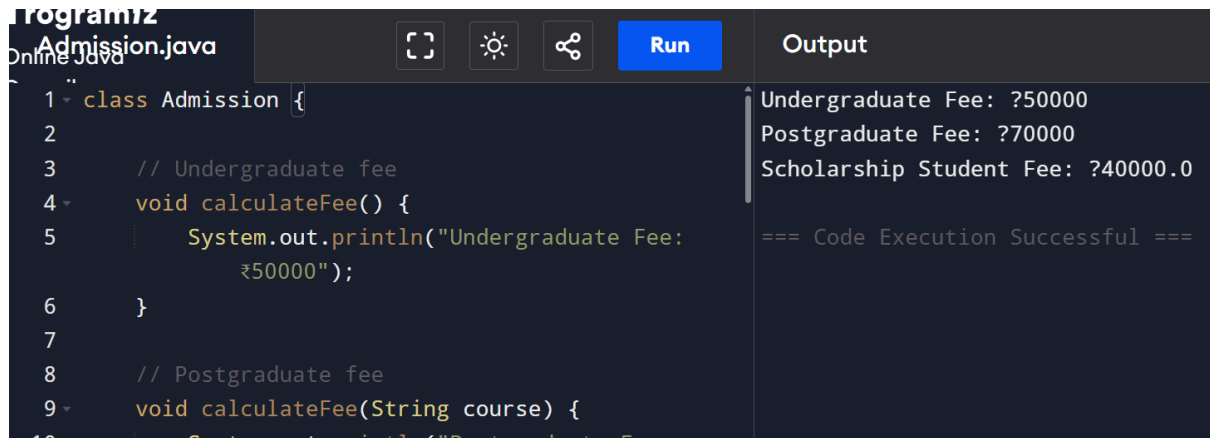
    // Scholarship student fee
    void calculateFee(String course, double scholarship) {
        double fee = 50000 - scholarship;
        System.out.println("Scholarship Student Fee: ₹" + fee);
    }

    public static void main(String[] args) {

        Admission a = new Admission();

        a.calculateFee();           // Undergraduate
        a.calculateFee("PG");       // Postgraduate
        a.calculateFee("UG", 10000); // Scholarship Student
    }
}
```

```
}
```



The screenshot shows an online Java IDE with a file named 'Admission.java'. The code defines a class 'Admission' with two methods: 'calculateFee()' which prints 'Undergraduate Fee: ₹50000', and 'calculateFee(String course)' which prints 'Postgraduate Fee: ₹70000' and 'Scholarship Student Fee: ₹40000.0'. The 'Run' button is highlighted in blue. The output panel on the right shows the execution results: 'Undergraduate Fee: ₹50000', 'Postgraduate Fee: ₹70000', 'Scholarship Student Fee: ₹40000.0', and a success message '=== Code Execution Successful ==='.

```
1 class Admission {
2
3     // Undergraduate fee
4     void calculateFee() {
5         System.out.println("Undergraduate Fee:
6             ₹50000");
7     }
8
9     // Postgraduate fee
10    void calculateFee(String course) {
11        System.out.println("Postgraduate Fee:
12            ₹70000");
13        System.out.println("Scholarship Student Fee: ₹40000.0");
14    }
15 }
```

Output

```
Undergraduate Fee: ₹50000
Postgraduate Fee: ₹70000
Scholarship Student Fee: ₹40000.0
=== Code Execution Successful ===
```

6. Shape Area Calculator using Abstract Class & Polymorphism Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

```
abstract class Shape {
```

```
    abstract void area();
```

```
    public static void main(String[] args) {
```

```
        Shape[] shapes = new Shape[3];
```

```
        shapes[0] = new Circle(5);
```

```
        shapes[1] = new Rectangle(4, 6);
```

```
        shapes[2] = new Triangle(3, 8);
```

```
        for (Shape s : shapes) {
```

```
            s.area();
```

```
        }
```

```
    }
```

```
}
```

```
class Circle extends Shape {
```

```
    double radius;
```

```
    Circle(double radius) {
```

```
        this.radius = radius;
```

```
    }
```

```
    void area() {
```

```
        System.out.println("Area of Circle = " + (3.14 * radius * radius));
```

```
    }
```

```
}
```

```
class Rectangle extends Shape {
```

```
    double length, width;
```

```
    Rectangle(double length, double width) {
```

```
        this.length = length;
```

```
        this.width = width;
```

```
    }
```

```
    void area() {
```

```
        System.out.println("Area of Rectangle = " + (length * width));
```

```
    }
```

```
}
```

```
class Triangle extends Shape {
```

```
    double base, height;
```

```
Triangle(double base, double height) {
```

```
    this.base = base;
```

```
    this.height = height;
```

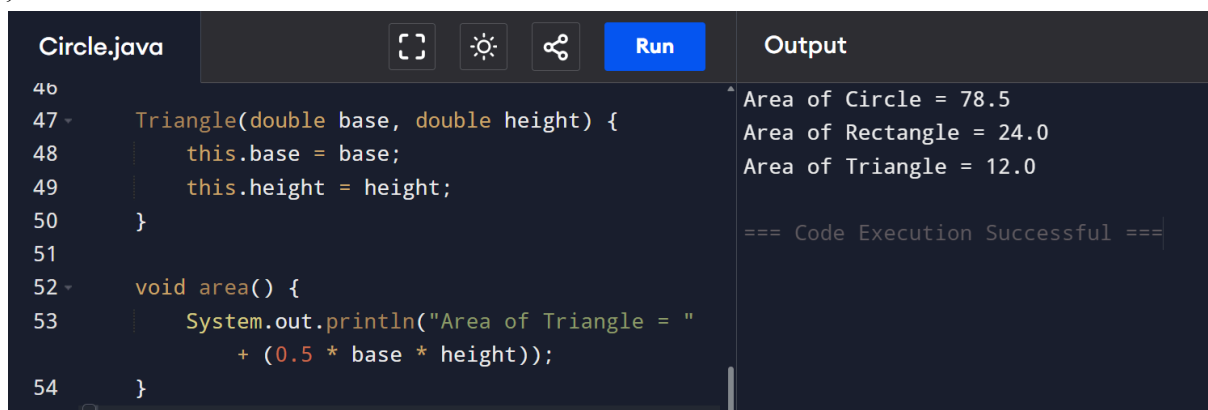
```
}
```

```
void area() {
```

```
    System.out.println("Area of Triangle = " + (0.5 * base * height));
```

```
}
```

```
}
```

A screenshot of a Java IDE interface. The left pane shows the source code for 'Circle.java'. The code includes a 'Triangle' constructor and an 'area()' method. The right pane shows the 'Output' window with the results of the program execution. The output displays the area of a circle (78.5), a rectangle (24.0), and a triangle (12.0), followed by a success message. The code in the left pane is as follows:

```
46
47 Triangle(double base, double height) {
48     this.base = base;
49     this.height = height;
50 }
51
52 void area() {
53     System.out.println("Area of Triangle = "
54         + (0.5 * base * height));
55 }
```

The output window shows:

```
Area of Circle = 78.5
Area of Rectangle = 24.0
Area of Triangle = 12.0

=== Code Execution Successful ===
```

7. Hospital Management System using Interfaces Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

```
interface MedicalRecord {
```

```
    void addRecord();
```

```
    void displayRecord();
```

```
    public static void main(String[] args) {
```

```
        MedicalRecord p = new Patient("Rahul", 101, "Fever");
```

```
        MedicalRecord d = new Doctor("Dr. Kumar", 201, "Cardiology");
```

```
p.addRecord();
p.displayRecord();

System.out.println();

d.addRecord();
d.displayRecord();
}
}

class Patient implements MedicalRecord {

    String name;
    int id;
    String disease;

    Patient(String name, int id, String disease) {
        this.name = name;
        this.id = id;
        this.disease = disease;
    }

    public void addRecord() {
        System.out.println("Patient Record Added");
    }

    public void displayRecord() {
        System.out.println("Patient Name: " + name);
    }
}
```

```
        System.out.println("Patient ID: " + id);
        System.out.println("Disease: " + disease);
    }
}
```

```
class Doctor implements MedicalRecord {
```

```
    String name;
    int id;
    String department;
```

```
    Doctor(String name, int id, String department) {
        this.name = name;
        this.id = id;
        this.department = department;
    }
```

```
    public void addRecord() {
        System.out.println("Doctor Record Added");
    }
```

```
    public void displayRecord() {
        System.out.println("Doctor Name: " + name);
        System.out.println("Doctor ID: " + id);
        System.out.println("Department: " + department);
    }
}
```



```
        this.department = department;
    }

    public void addRecord() {
        System.out.println("Doctor Record Added"
            );
    }

    public void displayRecord() {
        System.out.println("Doctor Name: " + name
            );
    }
}
```

```
Patient Record Added
Patient Name: Rahul
Patient ID: 101
Disease: Fever

Doctor Record Added
Doctor Name: Dr. Kumar
Doctor ID: 201
Department: Cardiology

=== Code Execution Successful ===
```

8. Online Shopping Order System using Packages Create two packages: ● shopping.product ● shopping.order Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

package shopping.product;

```
public class Product {

    int id;

    String name;

    double price;

    public Product(int id, String name, double price) {

        this.id = id;

        this.name = name;

        this.price = price;

    }

    public double getPrice() {

        return price;

    }

    public void displayProduct() {

        System.out.println("Product ID: " + id);

        System.out.println("Product Name: " + name);

    }

}
```

```

        System.out.println("Price: ₹" + price);
    }
}

```

The screenshot shows a Java IDE with a dark theme. On the left, the code for the `Product` class is visible. On the right, the output of the program is displayed, showing the details of an order placed successfully.

```

class Product {
    private int id;
    private String name;
    private double price;

    Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
}

```

```

Order Placed Successfully
Product ID: 101
Product Name: Laptop
Price: ₹50000.0
Quantity: 2
Total Amount: ₹100000.0

=== Code Execution Successful ===

```

9. Student Result Analyzer using Arrays of Objects Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

```

class Student {

    int rollNo;

    String name;

    int m1, m2, m3;

```

```

    Student(int rollNo, String name, int m1, int m2, int m3) {

        this.rollNo = rollNo;

        this.name = name;

        this.m1 = m1;

        this.m2 = m2;

        this.m3 = m3;

    }

```

```

    int total() {

        return m1 + m2 + m3;
    }

```

```
}
```

```
double average() {  
    return total() / 3.0;  
}
```

```
String grade() {  
    double avg = average();  
    if (avg >= 90) return "A";  
    else if (avg >= 75) return "B";  
    else if (avg >= 50) return "C";  
    else return "F";  
}  
}
```

```
public class StudentResultAnalyzer {  
    public static void main(String[] args) {  
  
        Student[] s = {  
            new Student(1, "Alice", 90, 85, 95),  
            new Student(2, "Bob", 70, 75, 80),  
            new Student(3, "Charlie", 60, 65, 70)  
        };  
  
        Student topper = s[0];  
  
        for (Student st : s) {  
            System.out.println(st.rollNo + " " + st.name);  
            System.out.println("Total: " + st.total());  
        }  
    }  
}
```

```

        System.out.println("Average: " + st.average());

        System.out.println("Grade: " + st.grade());

        if (st.total() > topper.total())

            topper = st;

    }

    System.out.println("\nClass Topper: " + topper.name);

}
}

```

<pre> public class Student { int rollNo; String name; int m1, m2, m3; Student(int rollNo, String name, int m1, int m2, int m3) { this.rollNo = rollNo; this.name = name; this.m1 = m1; this.m2 = m2; } } </pre>	<pre> Roll No: 1 Name: Alice Total: 270 Average: 90.0 Grade: A Roll No: 2 Name: Bob Total: 225 Average: 75.0 Grade: B </pre>
--	---

10. Smart Home Device Controller using Hierarchical Inheritance Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

```

class Device {

    void turnOn() {

        System.out.println("Device is ON");

    }

    void turnOff() {

        System.out.println("Device is OFF");

    }

}

```

```
}

void powerConsumption() {
    System.out.println("Power Consumption: 0 W");
}
}

class Light extends Device {
    void powerConsumption() {
        System.out.println("Light Power: 20 W");
    }
}

class Fan extends Device {
    void powerConsumption() {
        System.out.println("Fan Power: 75 W");
    }
}

class AirConditioner extends Device {
    void powerConsumption() {
        System.out.println("Air Conditioner Power: 1500 W");
    }
}

public class Main {
    public static void main(String[] args) {

        Device d;
```

```
d = new Light();  
d.turnOn();  
d.powerConsumption();  
d.turnOff();
```

```
System.out.println();
```

```
d = new Fan();  
d.turnOn();  
d.powerConsumption();  
d.turnOff();
```

```
System.out.println();
```

```
d = new AirConditioner();  
d.turnOn();  
d.powerConsumption();  
d.turnOff();
```

```
}
```

```
}
```

```
1 class Device {  
2     void turnOn() {  
3         System.out.println("Device is ON");  
4     }  
5  
6     void turnOff() {  
7         System.out.println("Device is OFF");  
8     }  
9  
10    void powerConsumption() {  
11        System.out.println("Power Consumption: 0");  
12    }  
13 }
```

Device is ON
Light Power: 20 W
Device is OFF

Device is ON
Fan Power: 75 W
Device is OFF

Device is ON
Air Conditioner Power: 1500 W
Device is OFF